Course Code: ITA1439 Course Name: Ethical Hacking Slot: A

CO2 - ASSIGNMENT 2

Evaluation and Engineering of Footprinting, Reconnaissance and Social-Engineering-Awareness Workflows in an Authorized Assessment

Field	Details
Student Name	**Manoj Reddy**
Reg. No.	**192311171**
Nature of assignment	Individual | Duration: 1 week | Total marks: 50
CO2	Execute footprinting and social-engineering techniques (DNS analysis, competitive-intelligence gathering, attack simulations) to collect information for identifying potential security risks.
Method	Understand -> Predict -> Design -> Collect -> Analyse -> Evaluate -> Defend -> Validate

Table 1. Mark distribution and report map

Question	Assessed area	Marks	Section
Q1	Footprinting strategy design and evaluation	8	1
Q2	DNS footprinting and analysis workflow	7	2
Q3	Competitive-intelligence and awareness utility	7	3
Q4	Reconnaissance data validation and recovery	6	4
Q5	Hierarchical organization footprinting	7	5
Q6	Footprinting approach comparison	7	6
Q7	Integrated diagnose-redesign-validate challenge	8	7
TOTAL		**50**	Appendices A-C: commands, scripts, raw data

Scope, authorization and how the evidence was produced

Authorization statement. Every target in this report is a fictional, lab-owned asset: domains under `lab-corp.example`, `secure/leaky/misconf/minimal.lab-corp.example` (reserved example names, RFC 2606) and addresses from the documentation ranges 203.0.113.0/24, 198.51.100.0/24 and 192.0.2.0/24 (RFC 5737). No live system, real organization or real person was probed. All people, companies and job titles used in the social-engineering study (Q3) are invented. Zone transfers are only attempted against lab-owned name servers for which written permission is assumed to be on file.

How the numbers were produced (please read). The environment used to prepare this report had no network access and no `dig`/`whois` binaries, so the lab was implemented as a deterministic simulator (`lab_simulation.py`, fixed seed, Appendix B). The simulator holds the ground truth for each lab target, answers queries the way a real name server, search engine or archive would, and counts queries, records, false positives and out-of-scope contacts. Every table and graph below is computed from that simulator. The shell commands shown in the text are the exact commands to run against a real authorised lab (for example BIND 9 containers you own) to reproduce the same experiments; real tool output should be pasted into Appendix C before submission. Parameters such as 'a host appears in the wordlist with probability 0.45' are assumptions of the model and are stated wherever they matter. Timing is supporting evidence only; coverage, reliability and scope compliance are the primary evidence.

Q1 - Evaluate and Engineer a Footprinting Strategy (8 marks)

1.1 Strategy design and classification of techniques

The central design decision is that **no single source is trusted on its own**. The strategy first fixes scope and authorization (Step 0), then collects passively from independent sources, then performs the minimum active collection that is needed to confirm what the passive sources suggest, and finally reconciles the sources against each other before anything is reported (Figure 1).

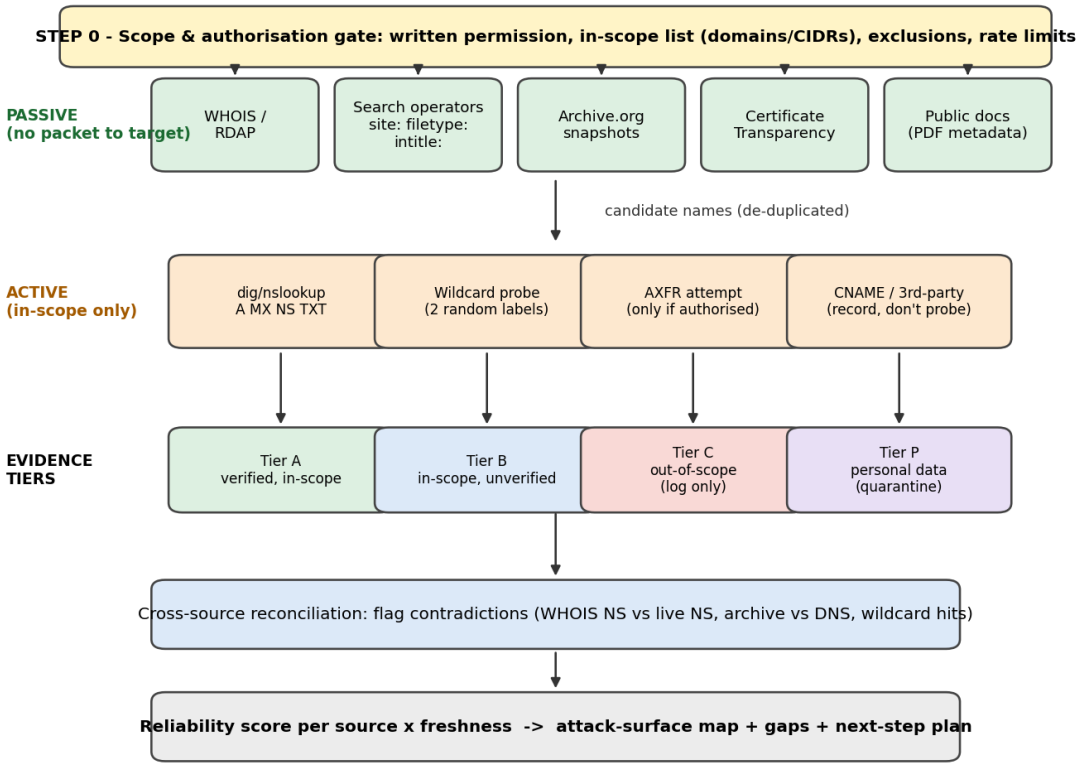


Figure 1. Multi-source footprinting strategy with scope gate, passive and active collectors, evidence tiers and reconciliation.

Table 2. Technique classification (passive vs active)

Technique	Tool / command	Class	Why	Yields
WHOIS / RDAP	`whois lab-corp.example`	Passive	Query goes to the registry/registrar, not the target	Registrar, dates, delegated NS, (redacted) contacts
Search-engine operators	`site: filetype:pdf intitle:"index of" inurl:login`	Passive	Query goes to the search engine only	Exposed pages, login portals, documents, directory listings
Archived pages	web.archive.org, `waybackurls`-style listing	Passive	Third-party archive	Removed pages, old hosts, old staff pages
Certificate Transparency	crt.sh query `%.lab-corp.example`	Passive	Public log, no contact with target	Host names that ever received a certificate
Public documents	download + `exiftool`	Passive*	*File is fetched from the target's web server (one ordinary GET)	Author names, internal paths, software versions
DNS A/MX/NS/TXT lookups	`dig`, `nslookup`	Active	Query reaches the target's authoritative server	Addresses, mail routing, SPF/DMARC, verification tokens
Wildcard probe	`dig random-9f3k.lab-corp.example`	Active	Two random labels to the authoritative server	Detects wildcard records that create false hosts
Zone transfer (AXFR)	`dig @ns1 lab-corp.example AXFR`	Active - intrusive	Only with explicit permission	Entire zone in one query

Technique	Tool / command	Class	Why	Yields
Brute-force sub-domains	wordlist + `dig`	Active - noisy	Hundreds of queries from one source	Hidden names not in any public source

1.2 Collection method and evidence handling

Collection was organised by source so that every item carries its origin. The command sequence for one target is:

```
# ---- Step 0: scope file (signed) ----          in_scope: lab-corp.example, *.lab-corp.example,
203.0.113.0/24
#                                                 excluded: payments.lab-corp.example, third-party
endpoints
# ---- PASSIVE ----
whois lab-corp.example                                    # WHOIS  -> source=WHOIS
curl -s 'https://crt.sh/?q=%25.lab-corp.example&output=json' | jq -r '.[].name_value' | sort -u   # CT
# search operators (typed into a search engine; record URL + date)
#   site:lab-corp.example            site:lab-corp.example filetype:pdf
#   site:lab-corp.example intitle:"index of"      site:lab-corp.example inurl:login
# Archive: https://web.archive.org/cdx/search/cdx?url=*.lab-
corp.example&output=txt&fl=original&collapse=urlkey
# ---- ACTIVE (in-scope only) ----
dig +noall +answer lab-corp.example A MX NS TXT
dig +short randomlabel-9f3k.lab-corp.example              # wildcard probe (twice with different labels)
dig @ns1.lab-corp.example lab-corp.example AXFR           # ONLY if authorised in writing
```

Every collected item is stored as `(source, item, timestamp, scope_status, verified?, personal_data?)` and placed into one of four tiers, so that out-of-scope, personal and unverified results are **never** mixed with verified in-scope findings:

Table 3. Evidence tiers

Tier	Meaning	Treatment
A	In scope and verified by a second independent source or direct authoritative answer	Used for attack-surface map and risk findings
B	In scope but single-source or unverified (archive snapshot, search snippet)	Reported as 'unverified'; re-checked before use
C	Out of scope (third-party SaaS endpoints, partner domains, SPF include targets)	Logged as 'seen, not touched'; never queried
P	Personal data (registrant names, e-mails, phone numbers, author names in metadata)	Quarantined; counted but not reproduced in the report body

1.3 Four contrasting authorised targets and prediction-versus-observation

Table 4. Construction of the four lab targets

Target	Design intent	Planted features
T1 secure	Well-protected domain	WHOIS privacy; 2 NS; SPF `-all`, DMARC `p=reject`; AXFR refused; no wildcard; clean PDF metadata
T2 leaky	Domain that leaks information	Registrant name/e-mail/phone in WHOIS; stale NS in WHOIS; `vpn`, `dev`, `mail` hosts; SPF `~all`; login portals and directory listing indexed; PDFs with author names, UNC paths, software versions; archive shows old `/admin`
T3 misconf.	Misconfigured DNS	AXFR open on ns1 (38 records); wildcard * record; dangling CNAME to a de-provisioned SaaS tenant; third NS is lame
T4 minimal	Minimal public footprint	One A record, one MX, 2 NS; WHOIS redacted; one indexed page; no archive; no documents

Predictions were written down for every source **before** collection. 'Observed' values are what the lab returned.

Table 5. Items per source - predicted / observed

Target	WHOIS	DNS	AXFR	SEARCH	ARCHIVE	DOCS	Total	|error|
T1	3 / 3	8 / 8	0 / 0	4 / 4	1 / 1	0 / 1	16 / 17	1
T2	7 / 7	9 / 10	0 / 0	20 / 27	3 / 5	4 / 6	43 / 55	12
T3	4 / 4	8 / 9	0 / 38	8 / 8	2 / 2	1 / 1	23 / 62	39

Target	WHOIS	DNS	AXFR	SEARCH	ARCHIVE	DOCS	Total	|error|
T4	3 / 3	4 / 4	0 / 0	2 / 1	1 / 0	0 / 0	10 / 8	2

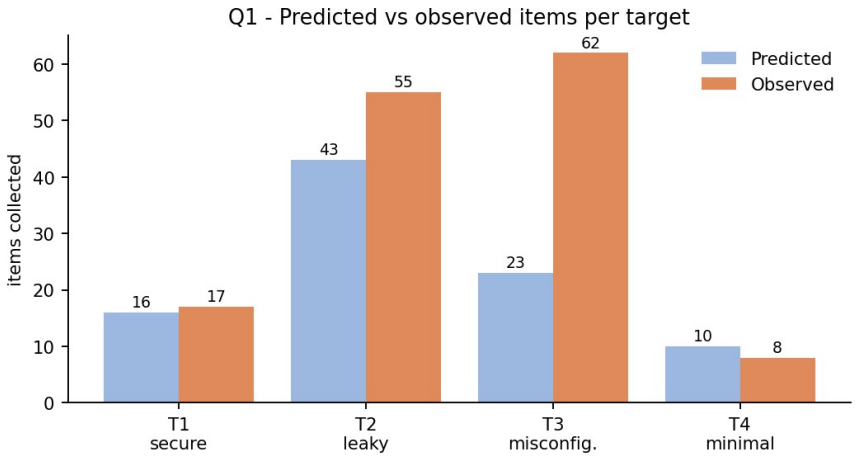


Figure 2. Predicted versus observed totals per target.

Interpretation. Prediction error rises with misconfiguration and leakage: T1 1, T4 2, T2 12 and T3 39 items. The best-protected targets were predicted almost exactly, while the leaky and misconfigured targets contained material the prediction missed. The biggest surprise was T3: I predicted that the zone transfer would be refused (the other DNS settings looked competent), but ns1 allowed it and returned 38 records versus 9 from ordinary queries - a 4x increase in knowledge from one query. For T4 the lesson is the opposite: 'minimal' did not mean 'empty'; the MX and NS records still revealed the mail and hosting providers, but the archive and document searches (predicted 1 and 0) returned nothing, so absence of results from one source was not evidence of absence.

Sources queried, records found and sensitive items

Table 6. Records found by source (selected)

Target	WHOIS	DNS	Search / archive / documents	Personal / sensitive
T1	Registrar, NS x2, dates; registrant REDACTED	A apex, A www, MX x2, NS x2, SPF `-all`, DMARC `reject`	4 marketing pages; 1 snapshot; 1 clean brochure	0 / 0
T2	Registrar, dates, org, NS (ns1/ns2.oldhost - stale); name, e-mail, phone	A x5 (apex, www, mail, vpn, dev), MX, NS x2, SPF `~all` `include:`, site-verification TXT	27 pages incl. 2 login portals + directory listing; 5 snapshots (old `/admin`, staff page); 6 documents with author names, `\\FS01\finance` paths, software versions	7 / 9
T3	Registrar, dates, 3 NS	9 via standard queries (wildcard * -> 203.0.113.9; CNAME `legacy` -> de-provisioned tenant); 38 via AXFR	8 pages; 2 snapshots; 1 document	0 / 14
T4	Registrar, dates, NS; registrant REDACTED	A apex, MX, NS x2	1 page; no snapshot; no documents	0 / 0

Contradictions between sources

Table 7. Cases where one source contradicted another

#	Target	Source A says	Source B says	Resolution / lesson
1	T2	WHOIS: name servers ns1/ns2.oldhost.example	Live DNS: ns1/ns2.leaky-dns.example	WHOIS delegation data is stale; the authoritative DNS answer wins (Tier A). WHOIS NS must not be queried.
2	T2	Archive (2023): `vpn` -> 203.0.113.40	DNS today: `vpn` -> 203.0.113.41	Host moved; archive is historical only. Report current value, keep old one as history.
3	T2	WHOIS org: 'LeakyCorp Ltd'	Document metadata company: 'Leaky Corp Pvt Ltd'	Naming mismatch; treat as same organisation only after confirming from the in-scope list.

#	Target	Source A says	Source B says	Resolution / lesson
4	T3	WHOIS: 3 name servers	Live DNS: only 2 answer (ns3 lame)	Parent/child mismatch = lame delegation finding.
5	T3	Search result lists `old.misconf` as a host	AXFR contains no such record	Name resolves only through the wildcard * -> **false positive**. Wildcard must be filtered.
6	T4	Archive: no snapshot	Search: 1 indexed page	Archive coverage is incomplete; absence in one source is not proof of absence.

1.4 Source-reliability evaluation

Table 8. Reliability of each source

Source	Reliability	Freshness	Misleading / outdated behaviour observed	Handling rule
Authoritative DNS (direct)	A	Seconds-TTL	Wildcard records manufacture hosts; split-horizon hides internal view	Primary truth for existence; always run wildcard probe
Recursive resolver	B	Up to TTL old	Cache returns old values after a change (see Q2 C6)	Never use as sole evidence of current value
WHOIS / RDAP (registry data)	B	Months	Delegated NS stale in T2	Use for registrar and dates only
WHOIS contacts	C	Years	Redacted or outdated; personal data	Tier P
Search engines	C	Weeks-months	Cached snippets of pages already removed	Tier B until fetched/confirmed
Archive.org	C (current state)	Historic	Shows hosts that no longer exist	History only
Certificate Transparency	B	Cert lifetime	Includes decommissioned hosts; wildcard certs hide names	Candidate generator, verify by DNS
Public documents	B content / C metadata	Document date	Metadata may be template residue	Tier P for names; Tier B for software versions

1.5 Changed condition: one domain -> fifty sub-domains and third-party services

The original strategy (S1) was applied literally: WHOIS, four search operators and one archive query, a 200-name brute-force wordlist, then six DNS queries (A, AAAA, MX, NS, TXT, CNAME) for **every** candidate, with no cache, no wildcard filter and third-party hosts treated like in-scope hosts. It was run against generated labs of 1, 5, 10, 25, 50 and 100 sub-domains (8% of them CNAMEs to third-party services, 10% stale names, wildcard DNS enabled), 30 random labs per size, and compared with a modified strategy (S2).

Table 9. Original (S1) versus modified (S2) strategy, mean of 30 labs per size

Sub-domains	Queries S1	Queries S2	Recall S1 / S2	False positives S1 / S2	Out-of-scope queries S1 / S2
1	1409	111	50% / 87%	200 / 0	0.0 / 0
5	1415	116	70% / 84%	198 / 0	0.0 / 0
10	1432	127	74% / 90%	197 / 0	2.0 / 0
25	1461	152	74% / 89%	190 / 0	4.6 / 0
50	1515	197	73% / 88%	180 / 0	9.3 / 0
100	1632	285	75% / 89%	159 / 0	18.4 / 0

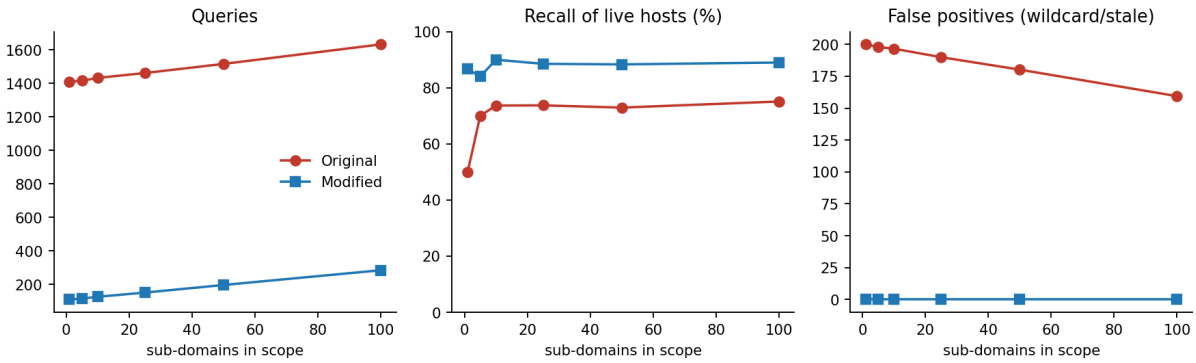


Figure 3. Effect of scale on the original and modified strategy.

Evaluation of S1. At 50 sub-domains S1 needs about 1515 queries, finds only 73% of the live hosts and reports about 180 false hosts, because every wordlist name resolves through the wildcard and each false name then receives six more queries. It also sends about 9 queries to third-party endpoints that are out of scope. The strategy does not fail because of the sources chosen; it fails because it has no wildcard handling, no de-duplication and no scope gate between 'candidate' and 'query'.

Modifications made (S2):

- Passive-first discovery: add Certificate Transparency to WHOIS, search and archive; build one de-duplicated candidate list before any active query.
- Wildcard probe (2 random labels) and filtering of answers equal to the wildcard address.
- Per-name verification limited to A and CNAME; zone-level records (SOA, NS, MX, TXT, CAA) queried once per zone instead of once per host.
- Scope gate for third-party services: the CNAME in our zone is recorded, but the provider's endpoint (Tier C) is never probed.
- Residual brute force reduced to the top 100 names, skipping names already known, with wildcard filtering.
- Cache of (name, type) answers so no query is sent twice.

Result and defence. For 50 sub-domains S2 uses about 197 queries (87% fewer), raises recall from 73% to 88%, removes all false positives and sends no out-of-scope queries. S2 therefore remains suitable for large-scale authorised reconnaissance, and its cost grows slowly (about 1.8 queries per extra host) while S1's cost is dominated by a fixed brute-force overhead. The canonical single lab (seed 192311171) gives the same picture: S1 1526 queries, recall 78%, 180 false positives, 6 out-of-scope queries; S2 192 queries, recall 86%, 0 false positives, 0 out-of-scope queries.

Limits that remain. About 12% of live hosts are still missed: they are neither in any passive source nor in the top-100 wordlist. Wildcard certificates reduce the value of CT logs, and S2 trusts the scope file: a wrong scope file would still lead to out-of-scope contact. S1 is still acceptable for a single small domain used for teaching, where seeing every step matters more than efficiency.

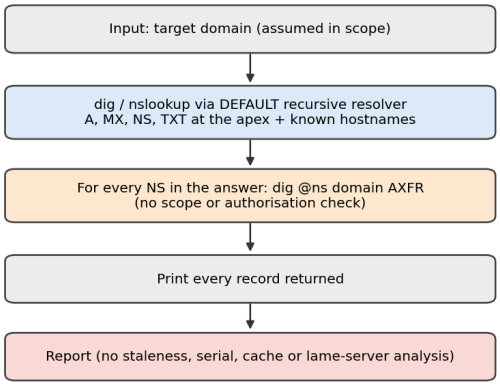
Table 10. Evidence checklist

Evidence expected	Where in this report
Footprinting strategy design	1.1
Authorised test targets	1.3
Collected-source records	1.2 commands; 1.3 records table
Prediction versus observation	1.3 (items per source, figure)
Source-reliability analysis	1.4
Scope analysis	1.5
Justified final strategy	1.5 (S2 and limits)

Q2 - Build a Reliable DNS Footprinting Workflow (7 marks)

2.1 The DNS footprinting workflow (W1)

W1 is the workflow most students would build first, using `dig` and `nslookup`. It is used here as the baseline that is evaluated against eight contrasting DNS cases; its weaknesses become the diagnosis in Q7.



W1 - baseline workflow evaluated in Q2 (limitations are diagnosed in Q7)

Figure 4. Baseline DNS footprinting workflow W1.

```
# A, MX, NS, TXT through the default resolver
dig +noall +answer lab-corp.example A
dig +noall +answer lab-corp.example MX
dig +noall +answer lab-corp.example NS
dig +noall +answer lab-corp.example TXT
nslookup -type=MX lab-corp.example                # same data with nslookup
# zone transfer - ONLY where authorised in writing
for ns in $(dig +short NS lab-corp.example); do
    dig @$ns lab-corp.example AXFR
done
```

2.2 How split-horizon, delegation and caching change what a tester sees

- **Split-horizon DNS.** The same name can have different answers depending on the client's source address. An external tester sees the external view (8 records in case C4) while an internal client sees many more (24 additional records). A report built only from the external vantage is accurate for the internet but understates internal exposure; a report that mixes two vantages without labelling them is misleading.
- **Delegation.** The parent zone hands a sub-domain (here `shop`) to another operator's name servers. The tester sees only a referral (NS records). Everything below the delegation belongs to the provider and is out of scope unless the contract says otherwise; querying or attempting AXFR there is a scope breach.
- **Caching.** A recursive resolver may serve an old answer until the TTL expires (case C6: 3600 s TTL, answer still shows the old address after the zone changed). Querying the authoritative server directly with `+norecurse` shows the current value; querying the resolver shows what clients may still be using. Both are useful, but they must be labelled.

2.3 Eight authorised DNS cases and predictions

Table 11. Contrasting DNS test cases (all lab-owned)

Case	Construction	Planted condition
C1	Correctly secured server	2 NS in sync; AXFR refused; only public records
C2	Zone transfer allowed	ns1 allows AXFR (40-record zone); ns2 refuses
C3	Stale records + secondary behind	`old-portal` A -> dead address; `legacy` CNAME -> de-provisioned SaaS (dangling); ns2 serial older than ns1
C4	Split-horizon	External view 8 records; internal view +24; AXFR refused externally

Case	Construction	Planted condition
C5	Delegation to out-of-scope third party	`shop` delegated to provider NS (out of scope; provider allows AXFR)
C6	Caching	A records changed; resolver caches old values for 3600 s
C7	Lame delegation	Parent lists ns3 but ns3 does not answer
C8	TSIG-protected transfer	AXFR requires a TSIG key; tester has none

Table 12. Prediction versus observation (W1) against verified lab ground truth

Case	Prediction (before running)	Observed with W1	Verified ground truth (issues)
C1	5 records; AXFR REFUSED; none	5/5 correct; AXFR refused; flagged: none	none
C2	9 records; AXFR REFUSED; none	9/9 correct; AXFR OPEN (40 rec.); flagged: open_axfr	open_axfr
C3	7 records; AXFR REFUSED; 1 stale record	6/7 correct; AXFR refused; flagged: none	dead_A, dangling_CNAME, serial_drift
C4	5 records; AXFR REFUSED; split-horizon	5/5 correct; AXFR refused; flagged: none	split_horizon
C5	6 records; AXFR REFUSED; delegation	6/6 correct; AXFR refused; flagged: none	oos_delegation
C6	5 records; AXFR REFUSED; resolver returns current values	3/5 correct; AXFR refused; flagged: none	none
C7	5 records; AXFR REFUSED; none	5/5 correct; AXFR refused; flagged: none	lame_ns
C8	5 records; AXFR REFUSED (no key); none	5/5 correct; AXFR refused; flagged: none	none

Differences and what they teach. (i) **C2:** I predicted REFUSED because the server looked correctly configured; ns1 returned the full zone, so a prediction made from 'looks secure' is not evidence. (ii) **C3:** I predicted one stale record; the lab contained three issues (dead A record, dangling CNAME with takeover risk, and a secondary running an older serial), and W1 flagged none of them. Worse, the resolver sometimes answered from the lagging secondary, giving `www` a wrong address. (iii) **C6:** I predicted current values; the resolver returned the old addresses for two records. (iv) **C7:** I predicted no issue; ns3 was lame and W1, which only used the resolver, never noticed. (v) **C5:** the standard-record count was right, but W1 followed the referral and sent queries to the provider's name server - a scope breach that the prediction did not consider.

Traced query returns (dig-style transcripts of the lab)

```
# C2  Zone transfer allowed  (lab-owned, written authorisation on file)
$ dig +short NS lab-corp.example
ns1.lab-corp.example.
ns2.lab-corp.example.
$ dig @ns1.lab-corp.example lab-corp.example AXFR
; <<>> DiG 9.18 <<>> @ns1.lab-corp.example lab-corp.example AXFR
lab-corp.example.   3600 IN SOA ns1.lab-corp.example. hostmaster.lab-corp.example. 2026101 7200 3600 1209600
3600
www.lab-corp.example. 3600 IN A 10.20.x.x   ; category=public
mail.lab-corp.example. 3600 IN A 10.20.x.x   ; category=public
mx1.lab-corp.example. 3600 IN A 10.20.x.x   ; category=public
ns1.lab-corp.example. 3600 IN A 10.20.x.x   ; category=public
ns2.lab-corp.example. 3600 IN A 10.20.x.x   ; category=public
autodiscover.lab-corp.example. 3600 IN A 10.20.x.x   ; category=public
webmail.lab-corp.example. 3600 IN A 10.20.x.x   ; category=public
smtp.lab-corp.example. 3600 IN A 10.20.x.x   ; category=public
vpn.lab-corp.example. 3600 IN A 10.20.x.x   ; category=infra
jenkins.lab-corp.example. 3600 IN A 10.20.x.x   ; category=infra
gitlab.lab-corp.example. 3600 IN A 10.20.x.x   ; category=infra
db01.lab-corp.example. 3600 IN A 10.20.x.x   ; category=infra
;; ... 28 more records ...
;; XFR size: 40 records
$ dig @ns2.lab-corp.example lab-corp.example AXFR
; Transfer failed.   (status: REFUSED)

# C6  Caching
$ dig @192.0.2.53 lab-corp.example A         # recursive resolver
lab-corp.example. 2217 IN A 198.51.100.10      <-- OLD value, 2217 s of TTL left
$ dig @ns1.lab-corp.example lab-corp.example A +norecurse
lab-corp.example. 3600 IN A 198.51.100.77     <-- authoritative, current

# C5  Delegation to out-of-scope third party
$ dig @ns1.lab-corp.example shop.lab-corp.example NS
shop.lab-corp.example. 3600 IN NS ns1.shopsaas-provider.example.   ; STOP: provider NS is out of scope

# C8  TSIG
$ dig @ns1.lab-corp.example lab-corp.example AXFR
; Transfer failed.   (status: NOTAUTH, TSIG key required)
```

```
# C7  Lame delegation
$ dig @ns3.lab-corp.example lab-corp.example SOA +time=3 +tries=1
;; connection timed out; no servers could be reached     <-- ns3 is lame
```

2.4 Does a zone transfer meaningfully change the attacker's knowledge?

The case-C2 zone has 40 records (weights: public 1, development 2, infrastructure 3, hidden/internal 4; total weight 106). Two attackers were compared: one with AXFR (1 query) and one without AXFR who uses 5 standard queries plus a 100-name brute-force wordlist (105 queries).

Table 13. Exposure by attacker method

Method	Queries	Records learned	Weighted exposure
AXFR (open zone)	1	40 / 40	100%
Standard queries + 100-name brute force	105	30 / 40	70%

Evaluation. AXFR gives the attacker 10 records that a wordlist does not find: `fin-sap-prd`, `hr-payroll01`, `bkp-nas-02`, `erp-uat3`, `dc01-corp`, `plc-gw-04`, `siem-core` and `dev6-dev8`. These names point at finance, HR, backup, directory, industrial-control and security-monitoring systems, i.e. exactly the high-value targets. The zone also exposes private (RFC 1918) ranges, Active-Directory service records and exact TTLs and serial numbers. So yes, the transfer meaningfully changes the risk: it replaces a noisy, partial, 105-query guessing exercise with a quiet, complete, single-query inventory. The honest limit is that an attacker with a good wordlist already learns about 70% of the weighted exposure, so closing AXFR is necessary but not sufficient.

2.5 Countermeasure redesign: restrict transfers, add TSIG, split the horizon

```
# primary (BIND 9) - generate the key once:   tsig-keygen -a hmac-sha256 xfer-key
key "xfer-key" { algorithm hmac-sha256; secret "<output of tsig-keygen>"; };
zone "lab-corp.example" {
    type primary;  file "db.lab-corp.example";
    allow-transfer { key xfer-key; };          // C: replace 'any' by ACL/IP;  D: require the TSIG
key
    notify yes;
};
# secondary:  server 203.0.113.53 { keys { xfer-key; }; };
# verification from the tester's host (must now fail):
dig @ns1.lab-corp.example lab-corp.example AXFR            # -> status: NOTAUTH / REFUSED
# E: internal names live only in the internal view (view "internal" { match-clients
{ 10.0.0.0/8; }; ... })
```

Table 14. Exposure after each countermeasure

Scenario	Attacker queries	Records visible externally	Weighted exposure
A Open AXFR (baseline)	1	40	100%
B Brute force only (AXFR refused)	105	30	70%
C allow-transfer ACL (secondary IP only)	105	30	70%
D ACL + TSIG key	105	30	70%
E ACL + TSIG + split-horizon (internal names moved)	105	13	17%

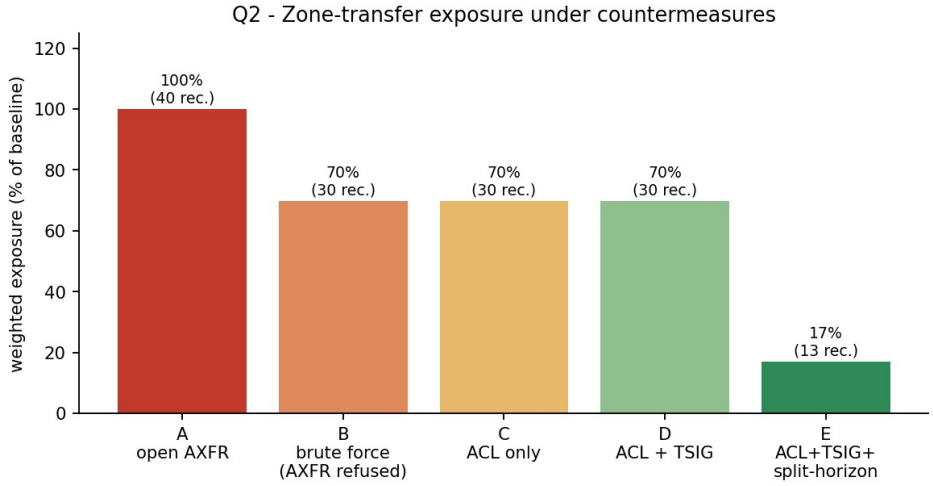


Figure 5. Reduction of external exposure after each countermeasure.

Restricting transfers (C) and requiring TSIG (D) remove the one-query inventory, so external exposure falls from 100% to 70% (the brute-force baseline). An ACL trusts the source address, whereas TSIG proves possession of a shared key, so D protects the secondary relationship even if the address is shared, NATed or compromised. The biggest reduction comes from split-horizon (E): moving infrastructure, hidden and directory names to the internal view cuts exposure to 17% because brute force can no longer find them externally. Residual risk: public names remain guessable, internal views must be protected from the internet, and the TSIG secret must be stored and rotated like any credential.

Table 15. Evidence checklist

Evidence expected	Where in this report
DNS workflow	2.1
Record analysis	2.2, traced transcripts
Authorised test cases	2.3 (eight cases)
Prediction versus observation	2.3 table and discussion
Zone-transfer risk evaluation	2.4
Countermeasure redesign	2.5

Q3 - Create a Competitive-Intelligence Awareness Utility (7 marks)

Purpose. A repeatable, modular workflow that collects public business information about an *authorised* organization and evaluates how that information could support phishing, vishing, pretexting or tailgating, so that awareness training and countermeasures can be designed. **No social-engineering attack is performed against any real person**; all organizations, people and job titles below are invented.

3.1 Workflow architecture and documented steps per source

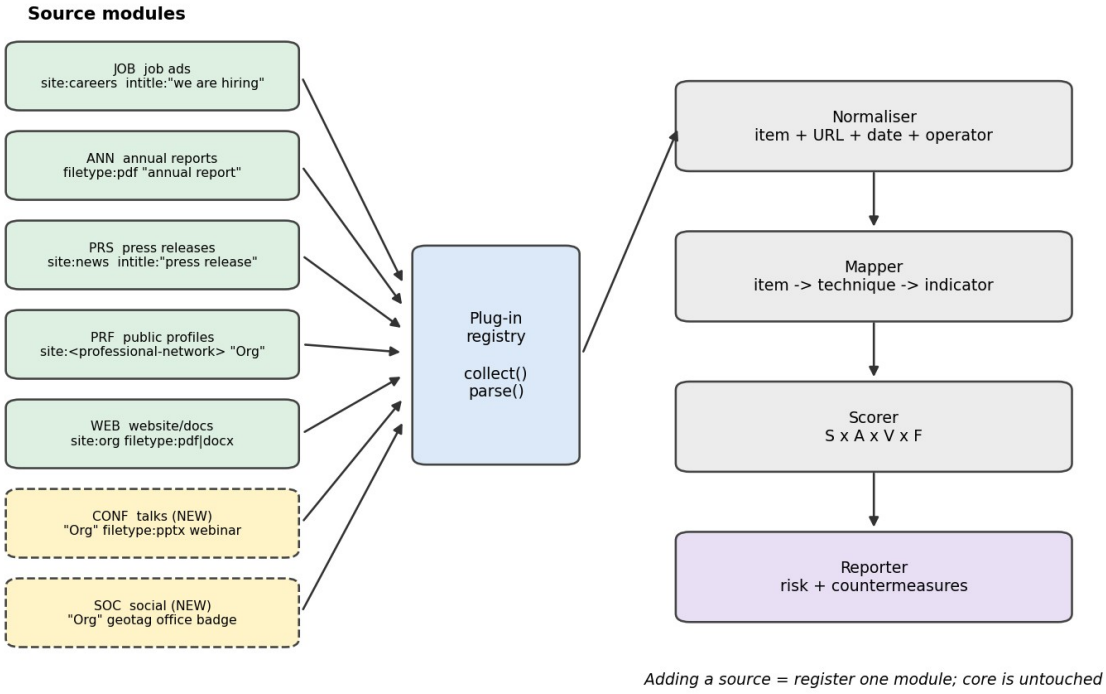


Figure 6. Modular competitive-intelligence and awareness workflow. Dashed yellow modules are the two sources added later.

```
class SourceModule:                    # interface every source implements
    name: str; operators: list[str]
    def collect(self, org) -> list[RawItem]      # runs the documented operators, records
URL+date+operator
    def parse(self, raw)   -> list[Item]         # Item(src, kind, text, url, date, operator)
REGISTRY = {}                                      # name -> module   (adding a source = one register()
call)
def run(org):
    items  = normalise([i for m in REGISTRY.values() for i in m.collect(org)])   # dedupe + provenance
    for i in items:
        i.tech, i.indicator = MAP[i.kind]                                        # item -> technique ->
indicator
        i.score = 100 * i.specificity * i.actionability * i.verifiability * i.freshness / 25
    return report(items, COUNTERMEASURES)
```

Table 16. Documented steps and search operators per source

Source	Documented steps	Operators (example org: orion-fin.example)	Fields recorded
JOB - job ads	List ads; extract technologies, reporting lines, locations, e-mail format; note date	`site:orion-fin.example intitle:"careers" "Orion Fintech" intitle:"we are hiring"`	URL, date, technology list, role, contact format
ANN - annual reports	Find report PDFs; extract executive names, vendors, addresses	`site:orion-fin.example filetype:pdf "annual report"`	URL, year, names (as roles), vendors, addresses
PRS - press releases	Collect releases; extract events, partners, office moves, dates	`site:orion-fin.example intitle:"press release" "Orion Fintech" "announces"`	URL, date, partner, location
PRF - public profiles	With client approval, note roles and visible badges; pseudonymise individuals	`site:<professional-network> "Orion Fintech"`	Role, tools mentioned, badge visible (yes/no)
WEB - site and	Crawl support/contact pages; download	`site:orion-fin.example`	URL, phone/e-mail, metadata

Source	Documented steps	Operators (example org: orion-fin.example)	Fields recorded
documents	documents and read metadata	`filetype:pdf OR filetype:docx site:orion-fin.example inurl:support`	author/path
CONF - talks (added)	Search conference and webinar slides and speaker bios	`"Orion Fintech" filetype:pptx webinar`	Speaker role, tools named
SOC - social (added)	Review company posts and geotagged office photos	`"Orion Fintech" office badge photo`	Photo content, location, onboarding posts

3.2 Item-to-technique mapping (Orion Fintech - rich information)

Each item is scored from four 1-5 ratings: **S**pecificity, **A**ctionability, **V**erifiability (0-1) and **F**reshness (0-1); `score = 100 x S x A x V x F / 25`. Source risk is the mean of the three highest item scores, classed High (>=60), Medium (30-59) or Low (<30).

Table 17. Collected items mapped to techniques and the indicator a target should notice

Src	Item collected	Technique(s) enabled	Indicator a target should notice	Score
JOB	Job ad: 'Experience with Okta SSO, Palo Alto firewalls, ServiceNow'	phishing, pretexting	Fake 'Okta session expired' mail from a non-corporate domain	80
JOB	Job ad: reports to 'Head of IT Service Desk', Pune office	vishing, pretexting	Caller claims to be from service desk and asks for MFA code	48
JOB	Job ad: recruiter e-mail format first.last@orion-fin.example	phishing	Mail from look-alike domain (orion-fln.example)	58
ANN	Annual report: new payment-processing vendor acquired (name + date)	phishing, pretexting	Unexpected 'vendor bank-detail change' request	38
ANN	Annual report: CFO and CEO names, board list	phishing	Urgent wire request 'from the CFO' bypassing process	43
ANN	Annual report: HQ address and 4 branch addresses	tailgating	Unbadged visitor following staff through a door	22
PRS	Press release: office move to Floor 7, opening date, visitor policy	tailgating, pretexting	Unknown 'contractor' without a visitor badge	34
PRS	Press release: partnership with a named payroll provider	phishing, vishing	Call from 'payroll provider' asking for staff data	38
PRF	Staff profile: 'Network admin, owns firewall and VPN rollout'	phishing, vishing, pretexting	Targeted spear-phish referring to VPN rollout	90
PRF	Profile photo: employee wearing badge, badge design visible	tailgating	Cloned-looking badge, wrong colour or no photo	51
PRF	Org chart reconstructed from 14 public profiles	pretexting, vishing	Caller names a real manager to build trust	52
WEB	filetype:pdf: brochure metadata with author name and internal path	pretexting	Request quoting an internal path or tool name	29
WEB	site: support page lists helpdesk phone and hours	vishing	Out-of-hours call insisting on urgency	36

3.3 Test organizations and prediction versus observation

Table 18. Test organizations

Organization	Public-information profile	Items collected
Orion Fintech	Rich: detailed job ads, annual report, press releases, many staff profiles, support page	13
Tiny Works	Sparse: one job ad, one founder profile, one generic contact e-mail	3
Nimbus Logistics	Misleading: old job ad, report of a different company with the same name, fake press release on a look-alike site, ex-employee profiles, impersonator profile	7

Table 19. Predicted human-risk level (before collection) versus observed

Organization	Source	Predicted	Observed score	Observed level	Match
Orion Fintech	JOB	High	62	High	yes
Orion Fintech	ANN	Medium	34	Medium	yes
Orion Fintech	PRS	Medium	36	Medium	yes
Orion Fintech	PRF	High	64	High	yes

Organization	Source	Predicted	Observed score	Observed level	Match
Orion Fintech	WEB	Medium	32	Medium	yes
Tiny Works	JOB	Low	5	Low	yes
Tiny Works	PRF	Low	13	Low	yes
Tiny Works	WEB	Low	8	Low	yes
Nimbus Logistics	JOB	Medium	1	Low	**no**
Nimbus Logistics	ANN	Medium	2	Low	**no**
Nimbus Logistics	PRS	Medium	11	Low	**no**
Nimbus Logistics	PRF	Medium	6	Low	**no**
Nimbus Logistics	WEB	Low	1	Low	yes

Result. 9 of 13 predictions matched (69%). All four misses are in the *misleading* organization, where I predicted Medium risk because information was plentiful. The observed risk was Low: unverifiable or outdated information (V and F near 0.1-0.4) is hard to turn into a convincing attack, and an attacker who uses it tends to reveal themselves (for example by quoting a retired system). The lesson is that **volume of public information is not the same as risk; verifiability and freshness matter**. Two caveats: a fake press release or an impersonator profile is itself evidence of an active attack and should be escalated even though its information score is low, and misleading information also threatens the *assessor*, because it pollutes the findings (this is why Q4 validates data).

3.4 Does the information meaningfully increase social-engineering risk?

The human-risk index (mean of source risks) is **45.8** for Orion Fintech, **8.5** for Tiny Works and **4.3** for Nimbus Logistics. Rich public information therefore raises the risk about 5.4 times compared with a sparse organization, and the 13 Orion items map to 7 pretexting, 6 phishing, 5 vishing and 3 tailgating opportunities (an item can enable more than one technique).

Most dangerous source: public staff profiles (PRF, 64) are the highest of the original five, with job ads (JOB, 62) a close second. The reason is that both combine high specificity (named tools, owned systems, role titles) with high actionability: the attacker learns *which tool to impersonate and who to address*. Annual reports (34), press releases (36) and website documents (32) are Medium: they supply names, vendors and addresses, which support business-e-mail-compromise and tailgating but give less technical detail.

3.5 Adding two new sources without restructuring the workflow

Two modules were added by registering them (`REGISTRY['CONF'] = ConferenceModule()` and `REGISTRY['SOC'] = SocialModule()`): conference/webinar material and company social posts with geotagged photos. The normaliser, mapper, scorer and reporter were not modified, which demonstrates the modular design.

Table 20. Effect of the two new sources on Orion Fintech

Source	Items	Source risk	Level	Main technique enabled
CONF	2	65	High	Phishing/pretexting with named security tools (e.g. EDR vendor lures)
SOC	2	53	Medium	Tailgating (badge reader model, door layout) and fake onboarding mail

Effect. The organization index rises from 45.8 to 49.5 (+8%), and the most dangerous source changes: conference material (CONF, 65) becomes the highest-risk source, ahead of profiles (64). A speaker's slide naming the security stack is more actionable than any job ad. This shows that the workflow must be re-run when new public channels appear.

3.6 Recommended countermeasures and their evaluated effect

Table 21. Awareness countermeasures (effect sizes are assumptions of a what-if model, not measurements)

Countermeasure	Addresses	Assumed residual risk	Awareness content / indicator trained
Enforced SPF/DKIM/DMARC + 'external sender' banner + phishing simulations	Phishing	60%	Look-alike domains, urgent payment or login requests
Call-back verification and MFA-reset policy (verify by a known number, never by the caller's)	Vishing	50%	Callers who push urgency or ask for codes
Out-of-band verification of vendor/bank changes; sanitised job ads and talk abstracts	Pretexting	55%	Requests quoting internal tools or names
Visitor escort, badge-display rules, anti-tailgating drills, no badge photos online	Tailgating	40%	Unbadged person following staff through a door

Table 22. Index with countermeasures added one group at a time (Orion Fintech, 7 sources)

Configuration	Human-risk index	Change vs no controls
No countermeasures	49.5	-
+ phishing and tailgating controls (2 countermeasures)	33.1	-33%
+ vishing and pretexting controls added (all 4, no restructuring)	25.1	-49%

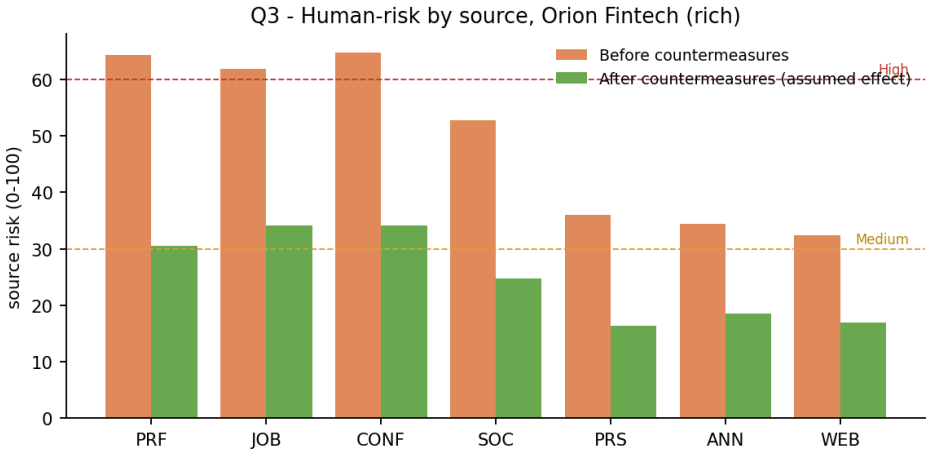


Figure 7. Source risk before and after the four countermeasures (Orion Fintech).

Records. Sources queried: JOB, ANN, PRS, PRF, WEB (+ CONF, SOC). Items found: 13 (Orion, original), 17 (Orion, after adding two sources), 3 (Tiny Works), 7 (Nimbus). Countermeasures recommended: the four groups above, with priority to controlling what job ads, profiles and talks reveal, because those are the sources that most increase risk.

Table 23. Evidence checklist

Evidence expected	Where in this report
Workflow architecture	3.1
Source operators	3.1
Item-to-technique mapping	3.2
Test organizations	3.3
Prediction versus observation	3.3
Risk evaluation	3.4-3.5
Recommended countermeasures	3.6

Q4 - Design a Reconnaissance Data-Validation Mechanism (6 marks)

4.1 Validation design

Every collected item (`value, type, source, age_days, source_reliability`) passes four checks. Two outcomes are final (**REJECT**, **DUP**), one is recoverable (**RECHECK**), and one means usable (**PASS**).

Table 24. Checks and outcomes

Check	Rule	Outcome if failed	Cost	Touches target?
Scope	Normalise (lower-case, strip trailing dot); in scope only if the name equals `lab-corp.example` or ends with `.lab-corp.example` **on a label boundary**, or the IP is inside 203.0.113.0/24, and it is not on the exclusion list (`payments.lab-corp.example`)	REJECT (out of scope) - final	1	No
Duplication	Key = (normalised value, type); keep first occurrence	DUP - merged, not lost	1	No
Freshness	age <= 30 d fresh; 31-365 d stale; > 365 d too old	31-365: RECHECK (re-query). >365: REJECT (unverifiable)	2	No
Source reliability	reliability >= 0.7 trusted; 0.4-0.7 needs confirmation; < 0.4 untrusted. Check performs a live confirmation lookup	0.4-0.7: RECHECK. <0.4: REJECT (unverifiable)	5	**Yes**

Recoverable vs rejected. *Recoverable* items are in scope but need a re-query or second-source confirmation (stale 31-365 days; reliability 0.4-0.7). *Rejected* items are out of scope (look-alike domains such as `lab-c0rp.example`, `notlab-corp.example`, `lab-corp.example.evil.test`, excluded hosts, third-party endpoints, IPs outside the CIDR) or cannot be verified (older than a year, source reliability below 0.4). A re-query is never sent for a rejected item.

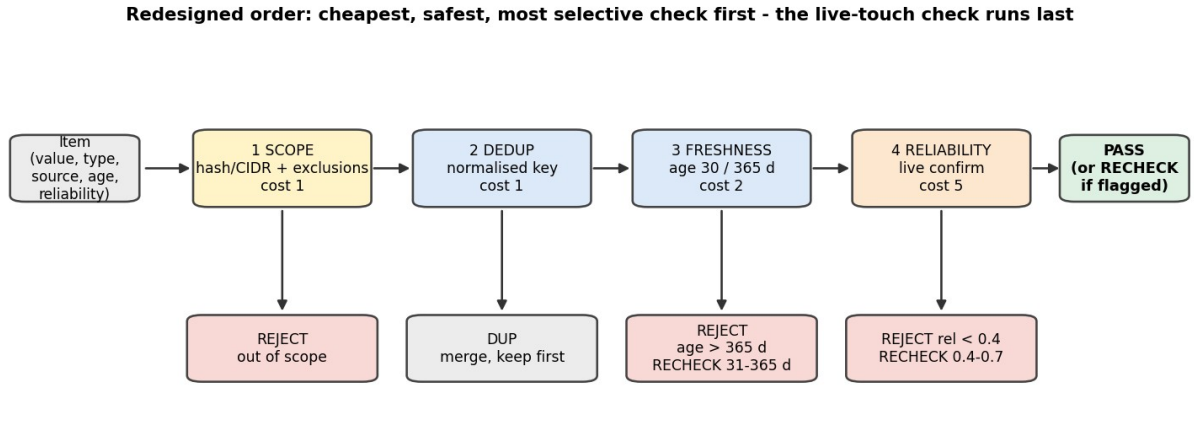


Figure 8. Redesigned validation pipeline: cheapest and safest checks first, live confirmation last.

```
def validate(item, scope, seen):
    if not in_scope(item.value, scope):          return REJECT("out-of-scope")       # gate: no
later check may run
    k = (norm(item.value), item.type)
    if k in seen:                                return DUP
    seen.add(k)
    if item.age > 365:                           return REJECT("too old")
    recheck = item.age > 30
    if live_confirm(item):                      # only reached by in-scope, unique, not-too-old
items
        if item.reliability < 0.4:               return REJECT("unreliable")
        recheck |= item.reliability < 0.7
    return RECHECK if recheck else PASS
```

4.2 Adversarial test sequences and prediction

Table 25. Six test sequences (20 items each)

Sequence	Construction
S1 all valid	20 unique, in-scope, fresh, reliable items
S2 out-of-scope heavy	8 valid + 12 out-of-scope (partner domain, `lab-c0rp.example`, `notlab-corp.example`, excluded `payments`, IP 198.51.100.7 and 203.0.114.9, suffix trick `lab-corp.example.evil.test`, third-party CDN)
S3 stale data	6 valid + 10 stale (90-180 d) + 4 older than a year
S4 duplicates	8 unique + 12 repeats (case variants, trailing dot, same record from another source)
S5 adversarial mixed	valid, out-of-scope **and** stale, excluded host **and** low reliability, high-reliability look-alike, duplicate out-of-scope pair, stale duplicate, in/out-of-CIDR IPs, trailing-dot duplicate, wildcard-style name
S6 mixed reliability	20 in-scope items with reliabilities 0.95, 0.6, 0.3, 0.8, 0.5 repeating

Table 26. Items PASS / RECHECK / REJECT / DUP - predicted, ground truth and observed (redesigned order)

Sequence	Predicted	Ground truth	Observed	Agree with truth
S1 all valid	20 / 0 / 0 / 0	20 / 0 / 0 / 0	20 / 0 / 0 / 0	100%
S2 out-of-scope heavy	8 / 0 / 12 / 0	8 / 0 / 12 / 0	8 / 0 / 12 / 0	100%
S3 stale data	6 / 10 / 4 / 0	6 / 10 / 4 / 0	6 / 10 / 4 / 0	100%
S4 duplicates	8 / 0 / 0 / 12	8 / 0 / 0 / 12	8 / 0 / 0 / 12	100%
S5 adversarial mixed	9 / 3 / 5 / 3	8 / 1 / 9 / 2	8 / 1 / 9 / 2	100%
S6 mixed source reliability	8 / 8 / 4 / 0	8 / 8 / 4 / 0	8 / 8 / 4 / 0	100%

Predictions matched for S1-S4 and S6, because each sequence had one dominant defect. S5 was predicted wrongly (9/3/5/3 versus the true 8/1/9/2): I classified items by their most visible defect, but out-of-scope status overrides every other property, so out-of-scope+stale, excluded+low-reliability, the high-reliability look-alike and both copies of the out-of-scope duplicate are all REJECT. The redesigned order reproduces the ground truth in every sequence.

4.3 Does the order of the checks matter?

Table 27. Three orders compared on all six sequences (120 items); effort units = sum of check costs

Order	Effort	Out-of-scope contacts	Accuracy
Original: reliability > freshness > dedup > scope	1026	18	98.3%
As listed in the brief: scope > freshness > dedup > reliability	830	0	100.0%
Redesigned: scope > dedup > freshness > reliability	808	0	100.0%

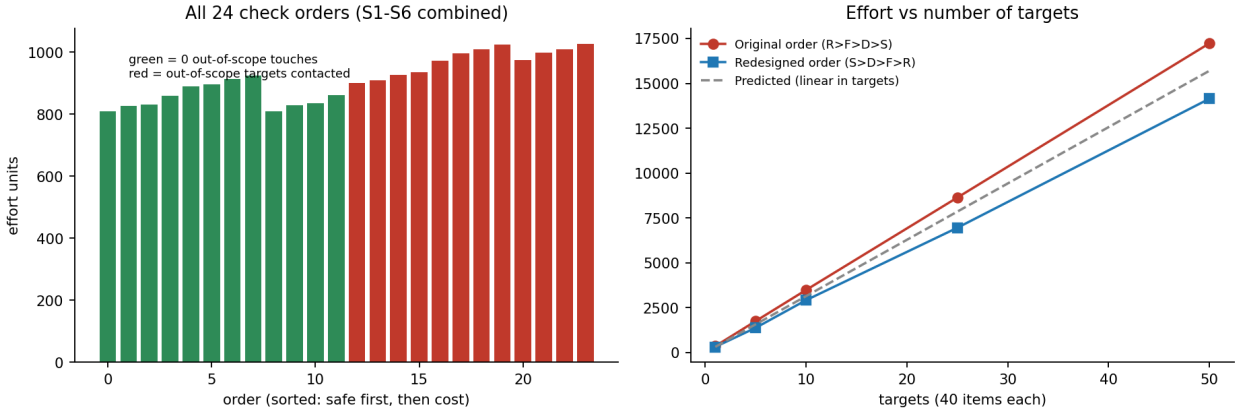


Figure 9. Left: all 24 orders (green = no out-of-scope contact). Right: effort as the number of targets grows.

Findings. The order changes both effort and correctness. (1) **Safety:** every order that runs the reliability check before the scope check performs live confirmations against out-of-scope targets (18 contacts across the six sequences) - a breach of the engagement rules; all 12 orders with scope before reliability have 0. (2) **Accuracy:** putting deduplication before scope labels out-of-scope duplicates as DUP instead of REJECT (accuracy 98.3%

instead of 100%). (3) **Effort:** among safe, 100%-accurate orders the cheapest is S>D>F>R at 808 units, 21% lower than the original order (1026) and 3% lower than the order listed in the brief (830), because duplicates are removed before the 2-unit freshness check and before the 5-unit reliability check.

Honest exception. In S6 (clean scope, no duplicates, mixed reliability) the original order is cheaper (164 vs 180 units) because rejecting low-reliability items early saves the later checks. Since the data mix is not known in advance and safety must not depend on luck, the redesigned order is still the right default; the exception shows that ordering is an engineering trade-off rather than a free win.

4.4 Redesign and justification

- **Scope first, as a hard gate.** It is the cheapest check (cost 1) and the only one whose failure is a breach, so nothing else may touch an item before it passes.
- **Then deduplication** (cost 1, very selective on repeated data), so no later check is paid twice.
- **Then freshness** (cost 2, purely local) to reject unverifiable ancient data before paying for a live lookup.
- **Reliability last** (cost 5, live): it runs only for items that are in scope, unique and young enough.
- RECHECK items are queued and re-queried only after the whole batch is validated, and only against in-scope targets.

4.5 Scalability inquiry: 1 to 50 targets

Prediction. Each target contributes 40 items (55% valid, 15% third-party/out-of-scope, 12% stale, 10% shared infrastructure repeated across targets, 8% low reliability). I predicted that effort grows linearly with the number of targets (the 1-target cost multiplied by N) and that the share of PASS/RECHECK/REJECT/DUP stays roughly constant.

Table 28. Outcomes and effort versus number of targets (redesigned order unless stated)

Targets	Items	PASS / RECHECK / REJECT / DUP	Predicted effort (linear)	Observed effort (redesigned)	Observed effort (original order)	Out-of-scope contacts (original)
1	40	26 / 7 / 5 / 2	314	314	354	4
5	200	107 / 31 / 47 / 15	1570	1375	1749	40
10	400	228 / 64 / 73 / 35	3140	2915	3483	55
25	1000	522 / 164 / 203 / 111	7850	6951	8633	159
50	2000	1079 / 326 / 412 / 183	15700	14143	17230	322

Analysis. Effort grows essentially linearly (7.1 units per item at 50 targets), but the observed cost at 50 targets (14,143) is 10% below the linear prediction (15,700) because shared infrastructure (common MX/NS records) is de-duplicated across targets by one global seen set. The original order costs 22% more and makes 322 out-of-scope contacts; the redesigned order makes none. About 16% of items need a re-query (326 at 50 targets), so re-query scheduling, not validation, becomes the dominant real-world cost; batching them by source and rate-limiting them is the next engineering step. Scope lookups should be compiled once (a set for names and a radix tree for CIDRs) so that the gate stays O(1) per item.

Table 29. Evidence checklist

Evidence expected	Where in this report
Validation design	4.1
Adversarial inputs	4.2
Pass/fail/re-check analysis	4.2 results table
Ordering evaluation	4.3
Justified redesign	4.4
Scalability inquiry	4.5

Q5 - Evaluate Footprinting of a Hierarchical Organization (7 marks)

5.1 Hierarchical footprint representation

The public footprint is a rooted tree: the parent domain is the root; sub-domains, departments and third-party services are descendants. Each node carries its scope status, so the traversal can decide *before* sending any query whether it may proceed.

Table 30. Node schema

Field	Meaning	Example
name	Unique identifier (FQDN or service name)	`crm.sales.lab-corp.example`
kind	domain / sub-domain / department / service	service
scope	`in` (authorised), `third` (third-party terminal), `out` (not authorised)	third
data	Number of public data items found when the node is queried	3
children	Child nodes discovered from this node's records (NS, CNAME, links)	[…]
alias	Optional pointer to another node (CNAME back-reference) - the source of cycles	`wiki -> sales`

5.2 Systematic traversal with an explicit boundary condition

```
D_MAX = 8
def footprint(root):
    queue, visited, report = [(root, 0)], set(), []
    while queue:                                        # breadth-first: level by level
        node, depth = queue.pop(0)
        # ---- explicit boundary condition (checked BEFORE any query) ----
        if depth > D_MAX:          stop(node, "depth-cap");     continue
        if node.scope != "in":     stop(node, node.scope);      continue   # out-of-scope or
third-party terminal
        if node.name in visited:   stop(node, "already-visited"); continue # cycle / alias guard
        visited.add(node.name)
        data = collect(node)                            # the only place a query is sent
        report.append((depth, node.name, data))         # record this level; empty data is a valid
result
        queue += node.children + ([node.alias] if node.alias else [])
    return report
```

Breadth-first order is used because it records the footprint level by level (parent, then its direct sub-domains, then departments, then services), makes the effort per level visible, and returns useful shallow results even if the engagement is stopped early. The three-part boundary condition guarantees termination (depth cap, visited set) and scope compliance (scope test) regardless of the shape of the tree.

5.3 Test structures

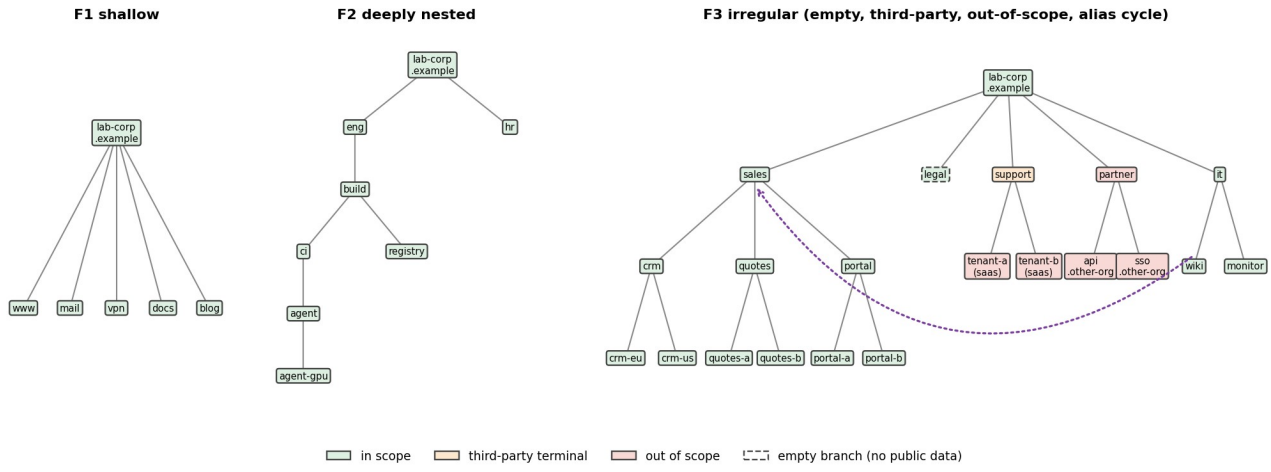


Figure 10. Shallow, deeply nested and irregular footprints. F3 contains an empty branch (`legal`), a third-party terminal (`support`), an out-of-scope partner and an alias cycle (`wiki` -> `sales`).

Table 31. Test structures

Footprint	Total nodes	In scope	Third-party	Out of scope	Max depth	Public items (in scope)
F1 shallow	6	6	0	0	1	15
F2 deeply nested	8	8	0	0	5	16
F3 irregular	21	15	1	5	3	28

5.4 Prediction versus observation

Before running the traversal I predicted, from the structure alone: nodes visited = number of in-scope nodes; boundary stops = number of third-party plus out-of-scope nodes that are children of in-scope nodes; items = public items of in-scope nodes.

Table 32. Nodes visited, boundary stops and items - predicted / observed

Footprint	Nodes visited	Boundary stops	Items	Queries sent	Nodes never touched (below a boundary)
F1 shallow	6 / 6	0 / 0	15 / 15	6	0
F2 deeply nested	8 / 8	0 / 0	16 / 16	8	0
F3 irregular	15 / 15	2 / 3	28 / 28	15	4

Verification. Shallow and deep footprints were predicted exactly. For the irregular footprint the visit count (15), the items (28) and the queries (15) matched, but the boundary stops were 3 instead of the predicted 2: the alias `wiki -> sales` pointed back to an already-visited node, and the visited-set guard stopped it (support (third), partner (out), sales (visited)). Without that guard the traversal would have looped. The 4 nodes below `support` and `partner` were never touched, which is the intended behaviour: the traversal does not even *discover* what sits beneath an out-of-scope or third-party boundary. Levels recorded for F3: L0: 1 nodes; L1: 3 nodes; L2: 5 nodes; L3: 6 nodes.

5.5 How depth and branching affect effort and completeness

Table 33. Nodes (= queries) in a balanced hierarchy with branching b and depth d

Branching b	d = 3	d = 5	d = 7	Widest level at d = 7
2	15	63	255	128
3	40	364	3,280	2,187
4	85	1,365	21,845	16,384

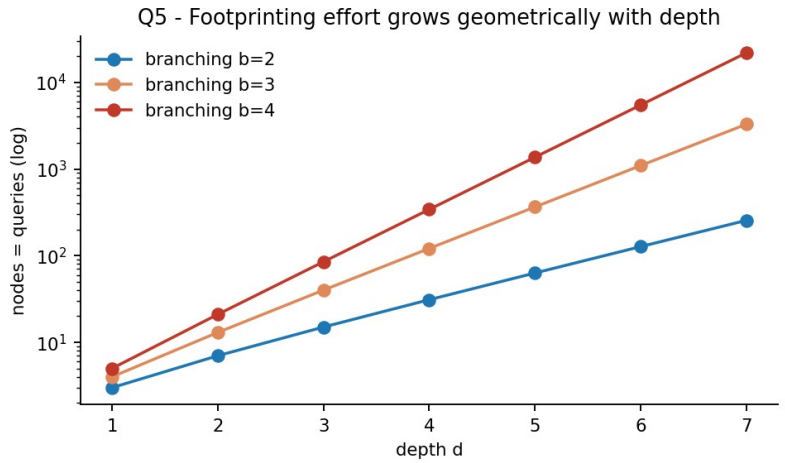


Figure 11. Effort grows geometrically with depth; the base of the growth is the branching factor.

Total effort is the geometric sum (b^(d+1) - 1)/(b - 1): doubling the depth does not double the effort, it squares it. Branching changes the shape: a bushy shallow tree (F1) is cheap and easy to complete; a deep narrow tree (F2) needs many levels but few queries per level; an irregular tree (F3) has most of its effort concentrated in a

few large branches (`sales` holds 10 of the 15 visited nodes). Completeness is limited by the depth cap: with b = 3 and d = 7 (3,280 nodes) a cap of 5 would visit only 364 nodes (11%), so the cap must be chosen from the scope document and any capped branch reported as *incomplete* rather than silently dropped. Queue length (the frontier) also grows as b^d, so very wide levels should be processed in rate-limited batches.

5.6 Empty branch and terminal nodes

`legal` is in scope but has no public data and no children. The boundary condition does not stop it (it is in scope and unvisited), `collect()` returns 0 items, it is recorded as a visited node with an empty result, and the queue simply has nothing to add. That is correct and different from an out-of-scope node: an empty branch is an *answer* ("we looked and found nothing"), while an out-of-scope node is *no query at all*. The report therefore distinguishes `visited, 0 items` from `skipped, out of scope`. The same loop also handles leaf nodes (no children) without special code.

5.7 Systematic versus ad-hoc approach

Table 34. Systematic BFS versus ad-hoc exploration (ad-hoc = mean of 300 simulated runs, same query budget)

Footprint	Coverage BFS	Coverage ad-hoc	Repeat visits BFS	Repeat visits ad-hoc	Out-of-scope contacts BFS	Out-of-scope contacts ad-hoc
F1 shallow	100%	87%	0	0.0	0	0.0
F2 deeply nested	100%	69%	0	0.0	0	0.0
F3 irregular	100%	73%	0	2.3	0	3.3

The ad-hoc tester picks promising names in no fixed order, has no visited list, overlooks 15% of links and notices a scope boundary 70% of the time (stated model assumptions). Even on the shallow footprint it misses about 13% of nodes because nothing tells the tester when the job is finished; on the irregular footprint it covers about 73%, repeats work (about 2.3 repeat visits per run) and touches 3.3 out-of-scope nodes on average. The systematic approach gives complete in-scope coverage, zero repetition, zero scope breaches and a record of every level, at the price of needing the tree (or at least its discovery rule) to be defined first. Ad-hoc exploration remains useful for creative discovery of *new* nodes that are then added to the tree.

Final design. BFS with depth cap, visited set and scope test before every query; empty results recorded explicitly; third-party and out-of-scope nodes recorded as boundary stops and never expanded; capped branches reported as incomplete.

Table 35. Evidence checklist

Evidence expected	Where in this report
Hierarchical representation	5.1
Traversal design	5.2 pseudocode
Test structures	5.3
Prediction versus observation	5.4
Boundary analysis	5.4, 5.6
Justified final design	5.7

Q6 - Evaluate Footprinting Approaches Under Constraints (7 marks)

6.1 Two approaches and what each produces

Table 36. Approach definitions

	A. Passive open-source collection	B. Active DNS and service footprinting
Techniques	WHOIS, Certificate Transparency, archive, search operators (4 base queries + one `site:<host>` per discovered host), manual review of documents	200-name brute force, PTR sweep of /24 blocks, then A, AAAA, MX, TXT, CNAME, NS, SRV, SOA for every host, and an A verification pass
Produces	Names, documents, people/roles, history; includes stale names	Live hosts, addresses, mail and service records; confirms what is alive now
Contact with target	None (queries go to search engines, logs, archives)	Every query reaches the target's DNS
Limiting factor	Search politeness (assumed 10 s per query) and manual document review (assumed 3 min per document, 0.7 documents per host)	Detection threshold (assumed IDS alert above 100 queries/minute) and wordlist coverage
Variants measured	Passive (semi-manual)	Active-fast at 20 queries/s; Active-throttled at 1.5 queries/s (stays below the alert threshold)

6.2 The two workflows (same authorised target)

```
# A. PASSIVE
whois lab-corp.example
$w.lab-corp.example A; done
crt.sh '%.lab-corp.example'
done      # PTR sweep
archive CDX 'lab-corp.example/*'
SRV SOA; do dig +short $h $t; done; done
site:/filetype:/intitle: operators (4)
# verification pass
site:<host> for each discovered host
exiftool on each downloaded document
queries (1.5 qps)
```

```
# B. ACTIVE
for w in $(head -200 wordlist.txt); do dig +short

for ip in 203.0.113.{0..255}; do dig +short -x $ip;

for h in $FOUND; do for t in A AAAA MX TXT CNAME NS

for h in $FOUND; do dig +short $h A; done


# throttled variant: add  sleep 0.67  between
```

6.3 Four-plus scope sizes - quantitative measures

Table 37. Passive versus active (mean of 10 labs per size). Recall = live hosts found; FP = stale/false names reported

Hosts	Passive queries	Passive time	Passive recall / FP	Active queries	Active-fast time	IDS alert (fast)	Active-throttled time	Active recall / FP
1	8	4 min	80% / 0	465	0 min	yes	5 min	100% / 0
10	16	20 min	83% / 1	533	0 min	yes	6 min	86% / 0
50	56	101 min	87% / 5	797	1 min	yes	9 min	76% / 0
250	248	8.2 h	86% / 25	2,636	2 min	yes	29 min	74% / 0
1,000	970	32.9 h	86% / 100	8,446	7 min	yes	94 min	63% / 0

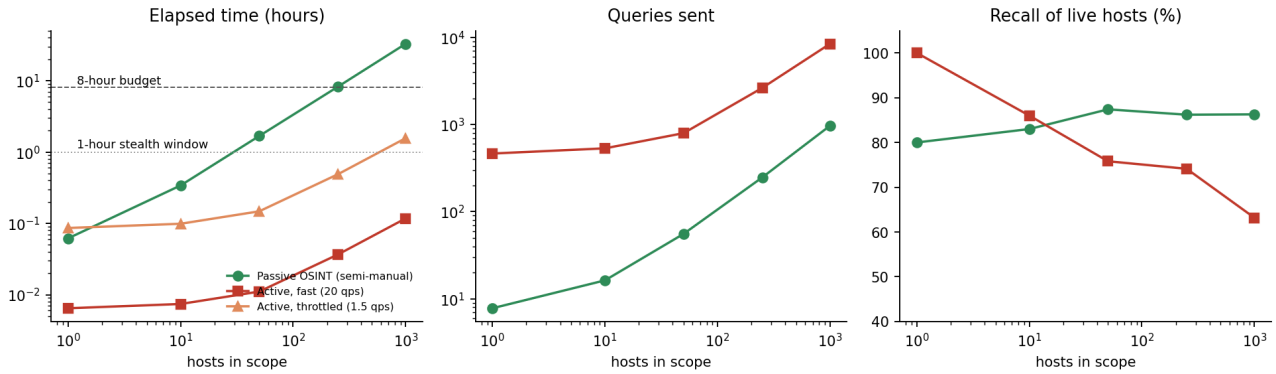


Figure 12. Time, queries and recall against scope size (log scales).

Where an approach becomes impractical. Semi-manual passive collection scales with analyst effort, not queries: it needs 1.7 h at 50 hosts but 8.2 h at 250 hosts, which exceeds an 8-hour reconnaissance budget, and 33 h at 1,000 hosts. Passive therefore stops being practical beyond roughly 250 hosts unless document review is

automated. Active collection is cheap in wall-clock time at full speed (7 min for 1,000 hosts) but it exceeds the alert threshold at every size; throttled to stay quiet it needs 1.6 h at 1,000 hosts, which is impractical inside a 1-hour stealth window but still inside an 8-hour day. Active coverage also falls with scope (recall 100% at 1 host, 63% at 1,000) because a fixed 200-name wordlist saturates, whereas passive recall stays near 86% but carries stale names (FP = 100 at 1,000 hosts).

6.4 Redundant and repeated collection

Table 38. Share of redundant work

Hosts	Passive: raw hits that repeat a host already found	Active: repeated (name, type) queries
1	48%	0.2%
10	37%	2.4%
50	37%	7.5%
250	36%	11.3%
1,000	35%	9.8%

Passive redundancy is high (35-48%): the same host is returned by the search operators, the archive and CT logs. This is useful as corroboration (a host seen by three sources is Tier A) but wasteful if each hit is analysed separately; the fix is to de-duplicate hosts before any analysis or document download. **Active redundancy** is small but real: the verification pass repeats every A query already sent in the brute-force and enumeration stages. A (name, type) cache would remove it. Wasted effort in active runs is more important than the percentage suggests because every wasted query is also a detectable query.

6.5 Is the faster or the quieter approach always better?

No. Active-fast is fastest but is the only approach that triggers an alert at every scope size, which is harmful when stealth or client operations matter and acceptable (even desirable) when the client wants the blue team tested or has deconflicted the test. Quiet is not free either: throttled active collection takes about 13 times longer than fast active collection at the same scope (e.g. 1.6 h versus 7 min at 1,000 hosts) and passive collection moves the cost to the analyst. The better engineering choice depends on whether the dominant constraint is time, detection risk or analyst effort, which is why the recommendation below depends on the scope and the context.

6.6 Defended recommendations

Table 39. Recommendation by scope and context

Context	Recommended approach	Justification from the measurements
Small scope (1-10 hosts), real engagement	Passive baseline (minutes), then active direct queries, fast, with the client's SOC informed	At 1-10 hosts passive takes under 21 min and finds about 80-83%; active adds live confirmation in under 30 s at 100% / 86% recall; alert risk is handled by deconfliction.
Large scope (250+ hosts)	Automated passive discovery (CT, archive, search) -> de-duplicate -> throttled active verification of the unique names only (the modified S2 strategy of Q1)	Manual passive is over budget at 250 hosts; fast active alerts and saturates at 63-74% recall; verification of ~86% passive recall plus throttling keeps queries low and quiet.
Teaching	Both, in this order: passive first, then active in a lab the class owns with a simple IDS	The measurable contrast (zero contact versus alert, recall versus stale names, time versus analyst effort) is the learning outcome; small scopes keep it cheap.
Repeated real assessments	Scripted hybrid: scope file, cache, wildcard filter, tiers, logs; throttle by client policy	Repeatability and audit matter more than speed; the cache removes the redundancy measured in 6.4 and the scope gate removes breaches.

Table 40. Evidence checklist

Evidence expected	Where in this report
Approach definitions	6.1
Two workflows	6.2
Quantitative measures	6.3
Redundancy analysis	6.4

Evidence expected	Where in this report
Defended recommendation	6.5-6.6

Q7 - Integrated Review: Diagnose, Redesign and Defend (8 marks)

Workflow selected: the DNS footprinting workflow W1 from Q2. It shows three of the limitations named in the brief at once: an unreliable source (a cached recursive resolver), poor ordering (zone transfer attempted before scope is checked) and a risk of leaving scope (queries sent to a third party's name servers).

A. Diagnose the limitation (2 marks)

Bottleneck and evidence. Running W1 on the eight lab cases of Q2 (47 probed records, 7 planted issues) produced:

- **Wrong data:** 3 of 47 reported values were wrong (C6: two cached addresses; C3: `www` returned by the lagging secondary).
- **Missed issues:** only 1 of 7 planted issues was flagged (the open zone transfer in C2); the dead A record, dangling CNAME, serial drift, split-horizon, lame NS and out-of-scope delegation went unnoticed.
- **Scope breach:** 3 queries were sent to out-of-scope servers (C5: two to chase the referral, one AXFR attempt at the provider's name server).

Underlying cause. (1) *Source unreliability*: the default resolver is a cache and may reach a lagging secondary, so its answers are not evidence of the zone's current state. (2) *Poor ordering*: AXFR is attempted on every name server listed, before any scope or authorization check. (3) *No verification step*: nothing compares name servers with each other or follows CNAMEs. Redundant collection is not the cause: W1 is the cheaper workflow.

B. Evaluation criteria (1 mark)

Table 41. Measurable criteria used to judge both workflows

Criterion	Definition	Target
Value accuracy (source reliability)	Reported values equal to authoritative truth / values probed	100%
Coverage of issues	Planted issues flagged / issues present (7)	100%
Scope compliance	Queries sent to out-of-scope servers	0
Queries	Total queries over the eight cases	Minimise, but not at the expense of the three above
Time	Supporting only: 0.04 s per query, 3 s per timeout	Report, do not decide on it

C. Redesign and trade-offs (2 marks)

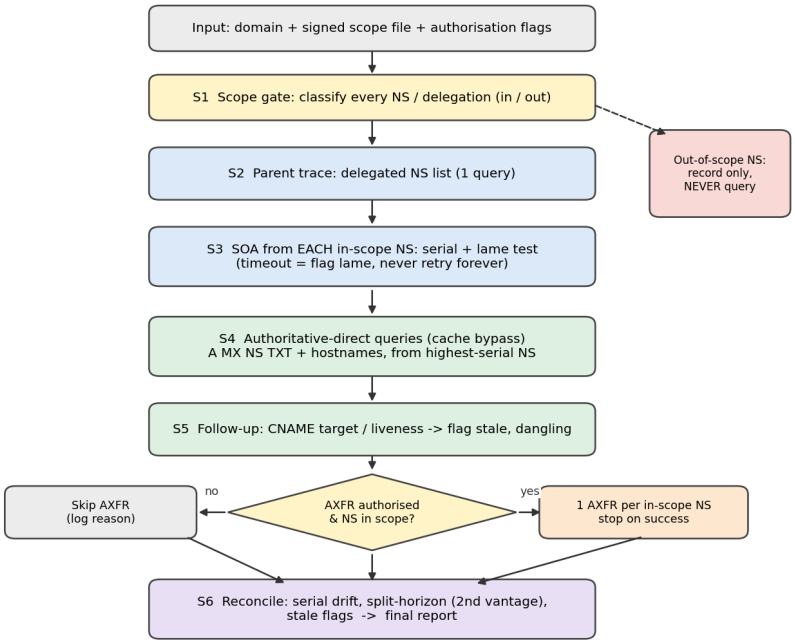


Figure 13. Redesigned workflow W2: scope gate, authoritative-direct queries, cross-checks and conditional AXFR.

Design logic (ordered steps).

1. **S1 Scope gate** - classify every delegated name server as in or out of scope before any query; out-of-scope servers are recorded, never queried (fixes the breach).
2. **S2 Parent trace** - one query to the parent for the delegated NS list.
3. **S3 SOA from each in-scope NS** - compare serials (detects a lagging secondary) and test each server for lameness with a bounded timeout.
4. **S4 Authoritative-direct queries** - ask the highest-serial in-scope NS with `+norecurse` for A, MX, NS, TXT and the hostnames, bypassing resolver caches (fixes wrong values).
5. **S5 Follow-up** - follow CNAME targets and check liveness to flag dangling and dead records.
6. **S6 AXFR only if authorised** and only against in-scope servers, stopping at the first success; with an authorised second vantage, compare answers to detect split-horizon; reconcile and report.

Trade-offs introduced. W2 sends more queries (SOA per server, CNAME follow-ups, a second-vantage check); it needs a signed scope file, per-server authorization flags and, for split-horizon detection, access to a second vantage point; and it reports what the authoritative servers say rather than what ordinary clients may still see, so cache behaviour is no longer visible unless a resolver cross-check is added deliberately.

D. Validate and compare under equivalent conditions (2 marks)

Table 42. W1 / W2 on the same eight cases and the same 47 probes

Case	Queries	Correct values	Issues flagged	Out-of-scope queries	Time (s)
C1	7 / 10	5 / 5 of 5	0 / 0 of 0	0 / 0	0.28 / 0.40
C2	11 / 13	9 / 9 of 9	1 / 1 of 1	0 / 0	0.44 / 0.52
C3	9 / 14	6 / 7 of 7	0 / 3 of 3	0 / 0	0.36 / 0.56
C4	7 / 11	5 / 5 of 5	0 / 1 of 1	0 / 0	0.28 / 0.44
C5	11 / 10	6 / 6 of 6	0 / 1 of 1	3 / 0	0.36 / 0.40
C6	7 / 10	3 / 5 of 5	0 / 0 of 0	0 / 0	0.28 / 0.40
C7	8 / 11	5 / 5 of 5	0 / 1 of 1	0 / 0	3.28 / 3.40
C8	7 / 10	5 / 5 of 5	0 / 0 of 0	0 / 0	0.28 / 0.40
Total	**67 / 89**	**44 / 47 of 47**	**1 / 7 of 7**	**3 / 0**	**5.56 / 6.52**

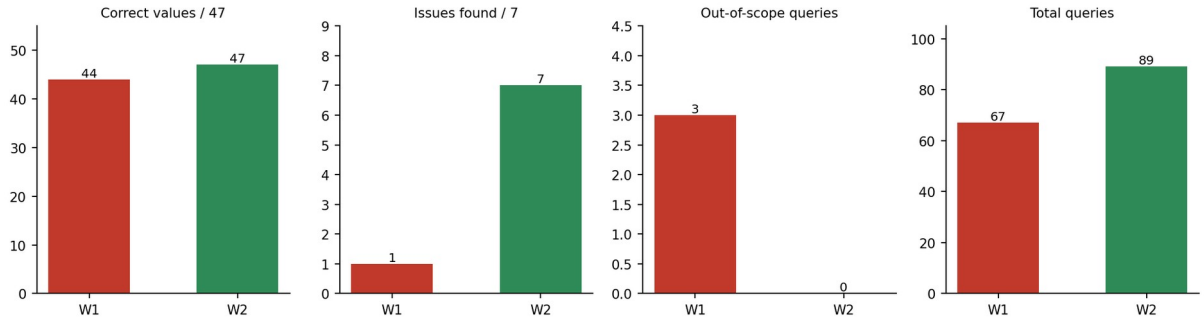


Figure 14. W1 (red) versus W2 (green) on the four criteria.

Result. W2 improves every criterion that matters for correctness: value accuracy 94% -> 100%, issue coverage 1/7 (14%) -> 7/7 (100%), out-of-scope queries 3 -> 0. The cost is 22 extra queries (+33%) and 0.96 s of extra time; the time difference is negligible and was not used to decide. The redesign therefore *does* improve the selected criteria; it does not win on query count, which is the intended trade.

E. Defend the final engineering judgment (1 mark)

Did the redesign meet its objective? Yes. The objective was reliable, scope-safe DNS footprinting; the comparison shows 100% accurate values, 7/7 issues found and zero out-of-scope contact, against 94% / 1-of-7 / 3 for W1, on identical inputs. The conclusion rests on coverage, accuracy and scope compliance, not on execution time.

When the original W1 may still be preferable. (i) A quick teaching demonstration of `dig`/`nslookup` basics, where the point is the commands rather than the interpretation. (ii) A single, lab-owned, single-server domain with no delegation, no secondary and no recent changes, where W1's answers equal W2's (case C1: both correct) at lower cost. (iii) When the goal is to see what ordinary clients currently receive through a cache: W1's resolver view is then the evidence, so the best practice is W2 plus one deliberate resolver query, clearly labelled.

Limits of this validation. The eight cases and the planted issues were designed by me, so W2 was tested on problems I knew to look for; the lab is simulated; and 100% issue coverage on seven planted issues is not a claim about real networks. The next step is to repeat the comparison on real authorised BIND deployments and on unplanned misconfigurations.

Table 43. Evidence checklist

Evidence expected	Where in this report
A. Bottleneck and cause	Section A
B. Measurable criteria	Section B
C. Redesign, logic, trade-offs	Section C
D. Quantitative comparison	Section D
E. Final judgment	Section E

Appendix A - Command sequences for reproducing the experiments on a real authorised lab

```
# Lab set-up (your own machines only): two BIND 9 containers for lab-corp.example, one resolver
docker run -d --name ns1 -p 5301:53/udp -p 5301:53/tcp -v $PWD/ns1:/etc/bind
internetsystemsconsortium/bind9:9.18
# Q1/Q2  footprinting and DNS
whois lab-corp.example ; dig +noall +answer lab-corp.example A MX NS TXT
dig @ns1.lab-corp.example lab-corp.example SOA +norecurse          # serial / lame test (W2 step S3)
dig @ns1.lab-corp.example lab-corp.example AXFR                      # only with written permission
dig @ns1.lab-corp.example lab-corp.example AXFR -y hmac-sha256:xfer-key:<secret>   # TSIG test
(authorised key only)
dig +short randomlabel-9f3k.lab-corp.example                          # wildcard probe
# Q3  search operators (record URL + date for each result)
#   site:orion-fin.example filetype:pdf        site:orion-fin.example intitle:"careers"
# Q4  validation:      python3 validate.py items.csv --scope scope.yaml --order scope,dedup,fresh,rel
# Q5  traversal:       python3 footprint.py tree.json --scope scope.yaml --dmax 8
# Q6  timing:          time ./passive.sh ; time ./active.sh --rate 20 ; time ./active.sh --rate 1.5
# Optional public practice target for zone transfers (published by its owner for training; check current
terms): zonetransfer.me
```

Appendix B - Scripts and data files delivered with this report

File	Content
`lab_simulation.py`	Deterministic lab simulator and all experiments for Q1-Q7 (seed 192311171)
`results.json`	Every measured value used in the tables and figures
`raw_output.txt`	dig-style transcripts of the lab cases (see also Q2.3)
`figs.py`	Generates all flowcharts, trees and graphs in this report

To regenerate: `python3 lab_simulation.py && python3 figs.py`.

Appendix C - Space for real tool output

Paste the real `whois`, `dig`, `nslookup`, `exiftool` and timing outputs from your own authorised lab here before submission, and update any number that differs from the simulated value.